
University of Kansas

**Arithmetic Expression Evaluator C++
Software Development Plan**
Version 1.5

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

Revision History

Date	Version	Description	Author
11/Feb/26	1.0	Added content for Sections 1.4, 1.5 and 2.1.	Yecolia E.
17/Feb/26	1.1	Added content for Sections 2.2, 2.3, and 2.4	Tugs-Oyut T.
17/Feb/26	1.2	Added content for Sections 1.1 & 1.2	Eric F.
18/Feb/26	1.3	Added content for Sections 4.7, 4.8, and 5. Also, updated some formatting and contact information on section 3.3.	John P.
20/Feb/26	1.4	Filled out sections 4.2.2, 4.2.3, and 4.2.4	Austin Haight
22/Feb/26	1.5	Filled out the rest of the sections and updated the document.	John P.

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Scope.....	4
1.3 Definitions, Acronyms, and Abbreviations.....	4
1.4 References.....	4
1.5 Overview.....	5
2. Project Overview.....	5
2.1 Project Purpose, Scope, and Objectives.....	5
2.2 Assumptions and Constraints.....	5
2.3 Project Deliverables.....	6
2.4 Evolution of the Software Development Plan.....	6
3. Project Organization.....	6
3.1 Organizational Structure.....	6
3.2 External Interfaces (N/A).....	6
3.3 Roles and Responsibilities.....	6
4. Management Process.....	7
4.1 Project Estimates (N/A).....	7
4.2 Project Plan.....	7
4.3 Project Monitoring and Control.....	8
4.4 Requirements Management (N/A).....	8
4.5 Quality Control.....	8
4.6 Reporting and Measurement (N/A).....	8
4.7 Risk Management.....	8
4.8 Configuration Management.....	9
5. Annexes.....	9

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

Software Development Plan

1. Introduction

1.1 Purpose

The purpose of the *Software Development Plan* is to create a document that contains all relevant information, and act as a guide for the C++ compiler project in EECS 348.

The following people use the *Software Development Plan*:

- The **Scrum Master** will use this document to help organize and schedule key parts of the project and ensure that all members are on the same page
- The **team members** will use this document to gain a better understanding of the project and learn more about their roles and what is expected of them as team members

1.2 Scope

This *Software Development Plan* will cover the intended plan that will be used for the C++ compiler project. The details of iterations will be included in the Iteration Plans. All plans are based on the requirements seen in the *Vision Document*.

1.3 Definitions, Acronyms, and Abbreviations

Currently, there are no definitions to be defined. More documentation (Project Glossary) will be updated if any are found.

- UPEDU – the Unified Process for EDUcation is the set of project artifact templates we will be using for this project's documentation.

1.4 References

The following documents and resources are referenced in this Software Development Plan:

1. Saiedian, Hossein:

EECS 348 Term Project in C++: *Arithmetic Expression Evaluator in C++*. Spring 2026.
University of Kansas, Department of Electrical Engineering and Computer Science.

2. Software Development Plan (SDP) Template:

Unified Process of EDUcation (UPEDU)

3. Project Vision:

The goal of this project is to design and implement a simple, accessible arithmetic expression interpreter that evaluates mathematical expressions correctly and efficiently. The system is intended to be clear and easy to use.

4. Project Glossary:

A glossary defining technical terms, abbreviations, and concepts used throughout the project documentation.

5. GitHub Repository – EECS348-project

Team repository containing source code, documentation, and version control

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

1.5 Overview

The Software Development Plan presents the structure, organization, and development approach for the Arithmetic Expression Evaluator in C++ project. The document explains how the project will be planned, developed, and completed using the Unified Process for EDUcation (UPEDU).

Project Overview – describes the purpose, scope, and objectives of the project. It explains the functionality of the arithmetic expression evaluator, including parsing and evaluating mathematical expressions according to operator precedence, and defines the expected deliverables of the project.

Project Organization – outlines the structure of the development team, including assigned roles and responsibilities for planning, implementation, testing, documentation, and configuration management.

Management Organization – describes the project schedule, major milestones, and development phases. It explains how the team will track progress, manage risks, ensure quality, and coordinate development activities throughout the project lifecycle.

Applicable Plans and Guidelines – summarizes the development practices and standards followed in this project, including the UPEDU process, version control using GitHub, documentation practices, and coding guidelines used during implementation.

2. Project Overview

2.1 Project Purpose, Scope, and Objectives

Purpose

The purpose of this project is to develop an arithmetic expression evaluator in C++ that can parse and evaluate mathematical expressions while correctly applying operator precedence, associativity, and parentheses.

The project also follows the Unified Process for EDUcation (UPEDU) to apply structured software engineering practices.

Scope

The system will support arithmetic expressions using numeric constants and the operators +, -, *, /, %, and **, including unary + and -. It will handle parentheses, provide a command-line interface, and include basic error handling for invalid expressions. The project includes required documentation, testing, and implementation.

Objectives

1. Implement a correct and functional arithmetic expression evaluator.
2. Handle operator precedence, parentheses, and unary operations properly.
3. Provide basic error detection for invalid input and runtime errors.
4. Follow the UPEDU development process and deliver a working, documented C++ program.

2.2 Assumptions and Constraints

- Project group size is 6 people.
- We are implementing our program following the UPEDU format
- The project must be fully completed by May 7th, 2026
- Our group is only implementing an arithmetic expression evaluator in C++ and nothing else

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

2.3 Project Deliverables

1. Project Plan | Delivery date: February 22nd
2. Software Requirements Specs | Delivery date: March 15th
3. Software Architecture Design | Delivery date: April 5th
4. Test Cases | Delivery date: May 7th
5. User Manual | Delivery date: May 7th

2.4 Evolution of the Software Development Plan

The *Software Development Plan* will be revised prior to the start of each Iteration phase. Changing our plan to adapt to any roadblocks/constraints that might show up in the future.

3. Project Organization

3.1 Organizational Structure

We function as a small team in which anyone can perform any role if necessary, though under normal circumstances section 3.3 will be accurate. Our performance will be reviewed regularly by ourselves and by Professor Saedian.

3.2 External Interfaces

This section is not currently applicable.

3.3 Roles and Responsibilities

Person	Unified Process for EDUcation Role
John Pannell - Email: j684p096@ku.edu - GitHub: jpannell890 - Expertise: Python, C, JavaScript, HTML	Programmer
Austin Haight - Email: austindhaight@ku.edu - GitHub: Austin-Haight - Expertise: C-based programming languages, Git project management	Repository Handler, Programmer
Tugs-Oyut Tsedensodnom - Email: t226t008@ku.edu - GitHub: TugsTS - Expertise: Python, C	Programmer
Eric Faltermeier - Email: e559f616@ku.edu - GitHub: Eric-Falt	Meeting Lead, Programmer
Alexander Wichern: - Email: alexanderwichern@ku.edu - GitHub: 876ajw06 - Expertise: Python, SQL	Notetaker, Programmer
Yecolia Ephraim:	Programmer, Notetaker

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

- Email: yecolia.ephraim@ku.edu - GitHub: yecoliaephraim-creator - Expertise: Python, HTML, C / C++	
---	--

Anyone on the project can perform [Any Role](#) activities.

For in-person availability, refer to this link: <https://www.when2meet.com/?34823193-8PZ5S>.

4. Management Process

4.1 Project Estimates

This section is not currently applicable.

4.2 Project Plan

Major milestone dates can be found in section 2.3. Project status will be reviewed weekly in our regular meetings. Meetings are tentatively scheduled at 5:00 PM on Tuesdays, though this may differ from week to week.

4.2.1 Phase Plan

This subsection is not currently applicable.

4.2.2 Iteration Objectives

Project Planning Phase:

- Form necessary team structures and delegate general tasks.

Software Requirements Phase:

- Develop robust, unambiguous specifications that a complete product must meet.
- Establish a programming conventions document.

Software Architecture Design:

- Organize product features based on level of integration required, for example: a GUI component of the product can largely be built around a functioning core in this case, whereas a syntax tree viewer must be considered during the core functionality iterations.
- Define how different parts of the software will interact with each other.

Functional Prototype:

- Build out a prototype that theoretically meets the software requirements.
- This phase can be completed in parallel to making test cases.

Test Cases:

- Build a set of tests that will verify if a piece of software meets our software requirements
- This phase can be completed in parallel to making a functional prototype

Release:

- Create a user manual and finalize the user interface components of the product.

4.2.3 Releases

The primary release will be a complete product, the scope and specificity of the project should not require a beta testing phase. A secondary release may be pursued after the primary to add additional features.

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

4.2.4 Project Schedule

As per section 2.3:

- Project Plan | Delivery date: February 22nd
- Software Requirements Specs | Delivery date: March 15th
- Software Architecture Design | Delivery date: April 5th
- Test Cases | Delivery date: May 7th
- User Manual | Delivery date: May 7th

4.2.5 Project Resourcing

This subsection is not currently applicable.

4.3 Project Monitoring and Control

4.4 Requirements Management

This section is not currently applicable.

4.5 Quality Control

Defects will be documented, tracked, and discussed in weekly meetings. Major quality control activities will be conducted at the end of each major milestone, including structured walkthroughs where the author presents their work and team members inspect for quality.

The deliverables must go through a rigorous review process, which includes creating checklists to confirm the work follows completeness, consistency, and alignment with past deliverables. The review's purpose is to confirm that each deliverable is of acceptable quality, using guidelines and checklists. Quality metrics, such as number of defects found per deliverable, will be collected to ensure the deliverables meet the expected standards.

Any defects found during review that are not corrected will be recorded in the meeting documentation and tracked as Change Requests to confirm resolution.

4.6 Reporting and Measurement

This section is not currently applicable.

4.7 Risk Management

Risks will be identified in the Inception Phase and during the planning and execution phases. Project risk is evaluated at least once per iteration and documented in this table, along with analysis for impact and likelihood. Mitigation strategies will be established to minimize high-priority risks.

ID	Risk Description	Probability	Impact	Mitigation Strategy	Status
R1	Improper implementation of operator precedence, including the right-to-left process with exponentiation.	Medium	High	Design operator precedence implementation carefully and consider all cases using unit tests for each operator.	Open

Arithmetic Expression Evaluator C++	Version: 1.5
Software Development Plan	Date: 22/Feb/26
EECS348-SDP-TermProj-v1.5	

R2	Division by zero would cause a crash in the program if not handled properly.	Medium	High	Use robust error handling and test cases to confirm the error will be caught.	Open
R3	Parentheses in expressions must be included as a factor to determine the order of evaluation.	Medium	High	Consider using a stack or tree to handle the order of evaluation with parentheses, while testing complex expressions with nested parentheses.	Open
R4	Numeric constants must not be misinterpreted, and a future request may be made to support floating-point numbers.	High	High	Use a design to implement tokenization and write unit test for integers and floats in preparation of a change.	Open
R5	Design and implementation do not fully align with requirements.	Low	High	Complete a complete review of the requirements before the coding and implementation begins.	Open

4.8 Configuration Management

Project artifacts will be properly versioned and tracked to ensure control of the development. For this project, Git will be used to manage version control with a repository available on GitHub. Members are expected to commit newly changed or edited work to be reviewed in weekly meetings.

All source code, test scripts, and data files are included in baselines. This includes C++ source files and test cases for each feature that use consistent naming (e.g., Project-Plan, Test-Cases, Design-Doc) and sequential versioning convention (1.0, 1.1, etc.). Documentation related to the source code is also included in the baseline, such as design documentation. A baseline will be produced after each major milestone or part of the development project. The final baseline will include deliverables that are ready for submission.

The Change Requests are reviewed and approved by the members of the project team before implementation.

5. Annexes

The project will follow the UPEDU process.

All coding will follow the team's Programming Guidelines, such as object-oriented programming principles, naming conventions, commenting and documenting all code meaningfully, and error-handling procedures.

Program design will follow the Design Guidelines, including using data structures (stacks or trees) to handle operator precedence and parentheses, organizing the system with a tokenizer, parser, and evaluator, and allowing for future extensions and error handling for clear results.

Other applicable process plans are listed in the references section.